

(19) **United States**

(12) **Patent Application Publication**

EVERSON

(10) **Pub. No.: US 2025/0280051 A1**

(43) **Pub. Date:**

**Sep. 4, 2025**

(54) **SELECTIVE PRUNING OF CANDIDATE LOAD-BALANCING SERVERS**

(71) Applicant: **Capital One Services, LLC**, McLean, VA (US)

(72) Inventor: **Richard EVERSON**, Mechanicsville, VA (US)

(21) Appl. No.: **18/594,705**

(22) Filed: **Mar. 4, 2024**

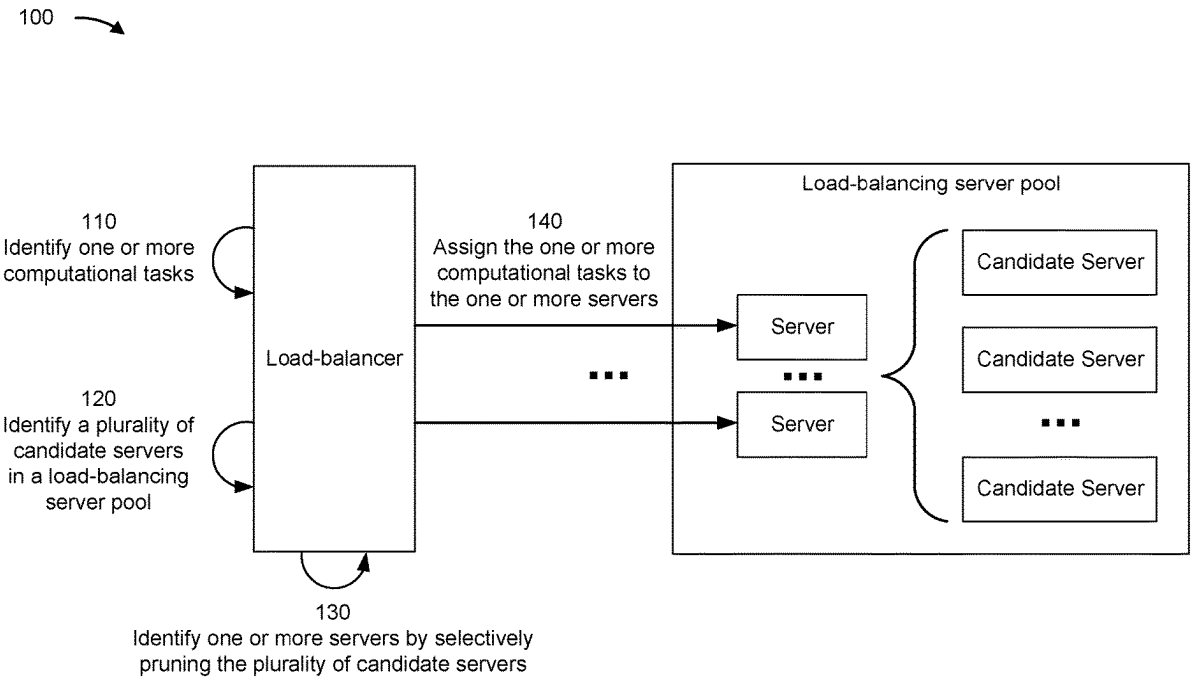
**Publication Classification**

(51) **Int. Cl.**  
**H04L 67/1012** (2022.01)  
**H04L 67/1008** (2022.01)

(52) **U.S. Cl.**  
CPC ..... **H04L 67/1012** (2013.01); **H04L 67/1008** (2013.01)

(57) **ABSTRACT**

In some implementations, a load-balancing system may identify one or more computational tasks. The load-balancing system may identify a plurality of candidate servers in a load-balancing server pool. The load-balancing system may identify one or more servers by selectively pruning the plurality of candidate servers. The load-balancing system may assign the one or more computational tasks to the one or more servers.



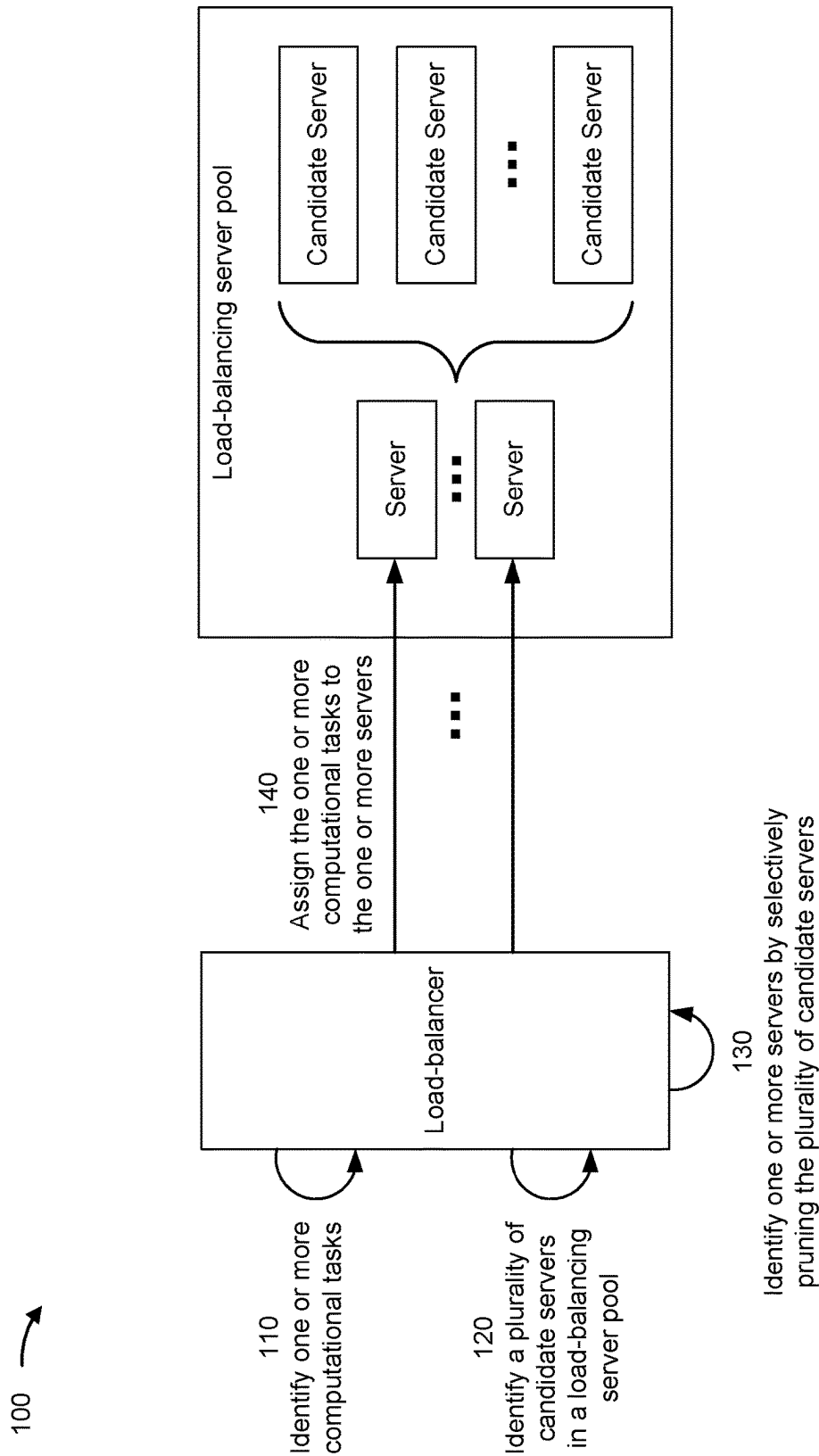


FIG. 1

200

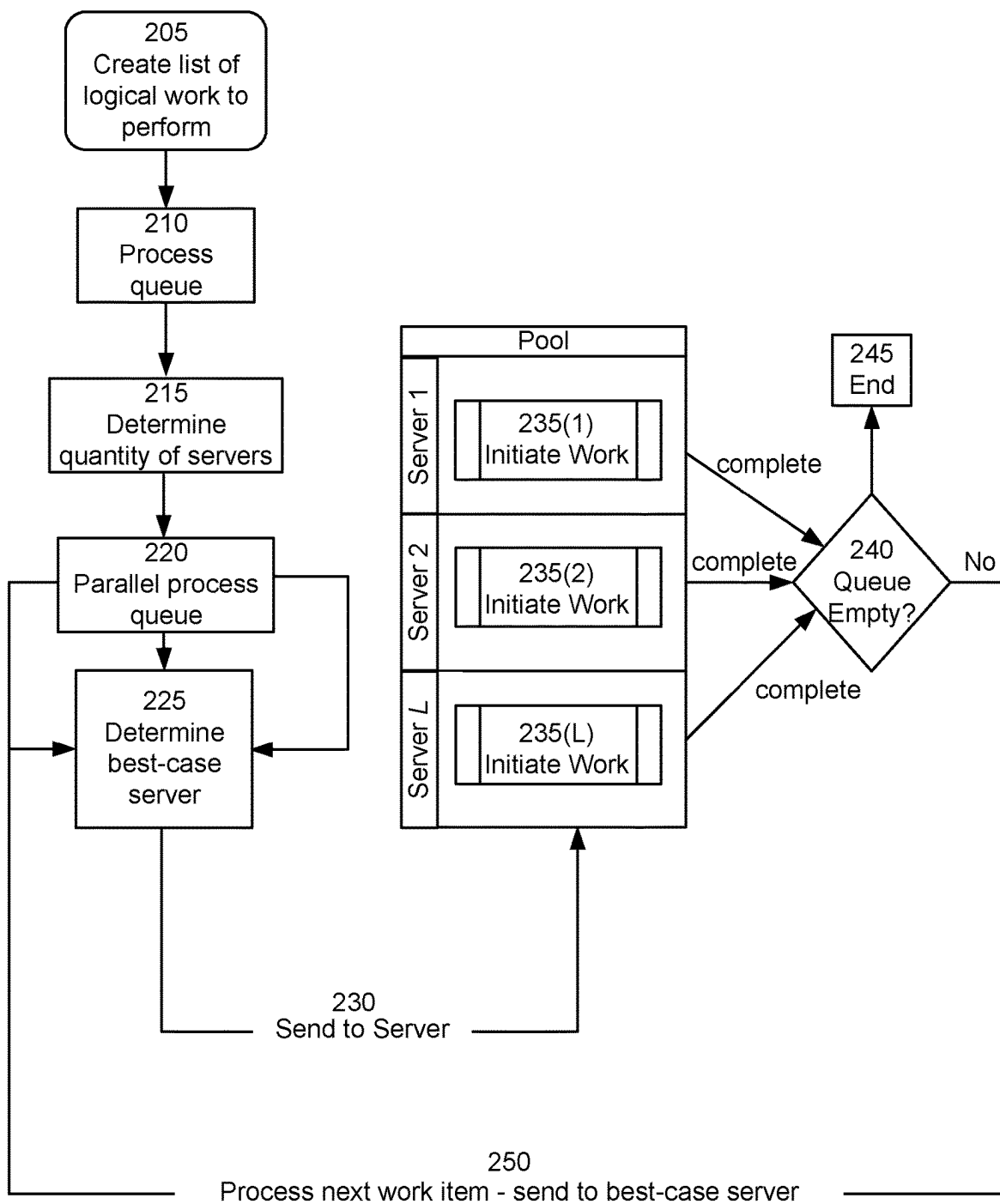


FIG. 2

300 ↗

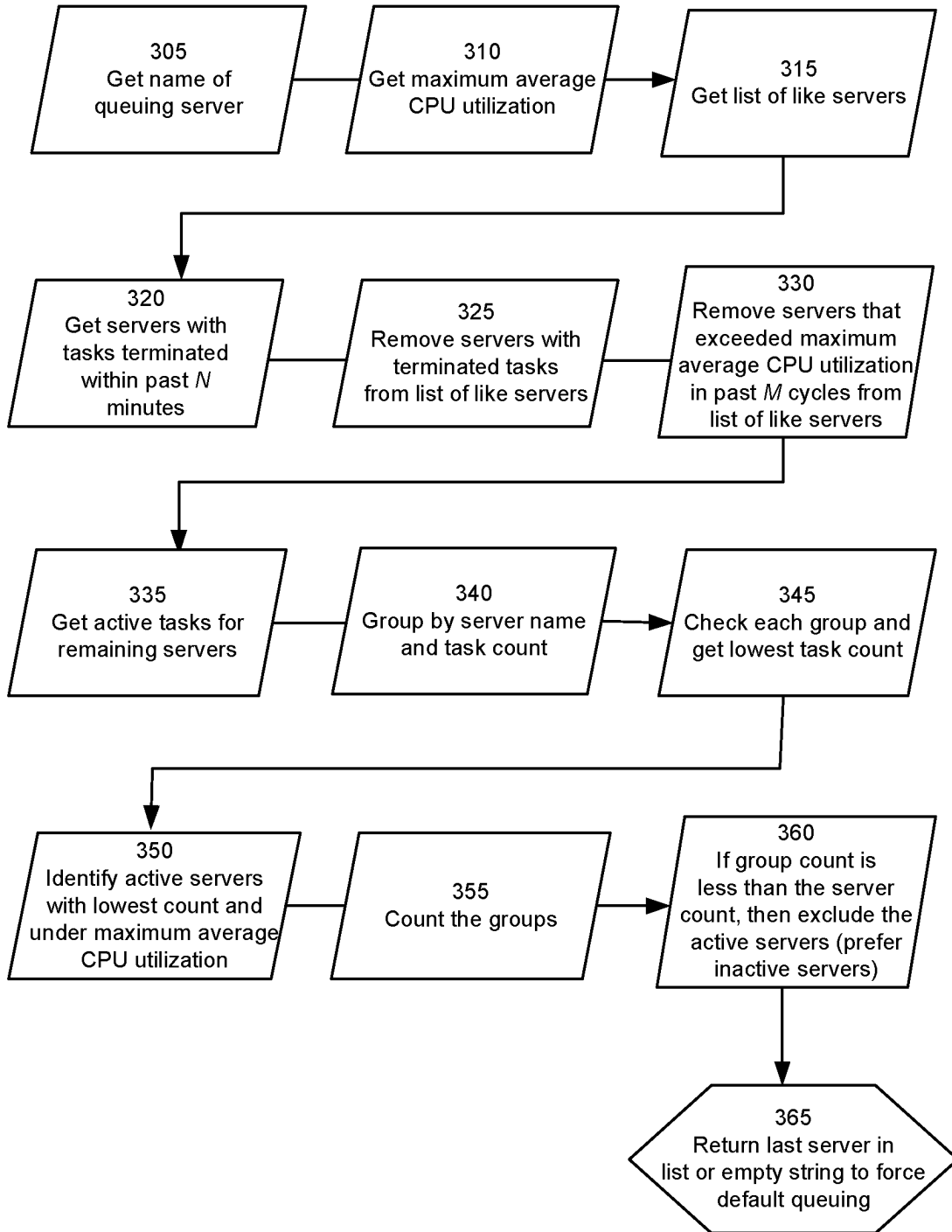


FIG. 3

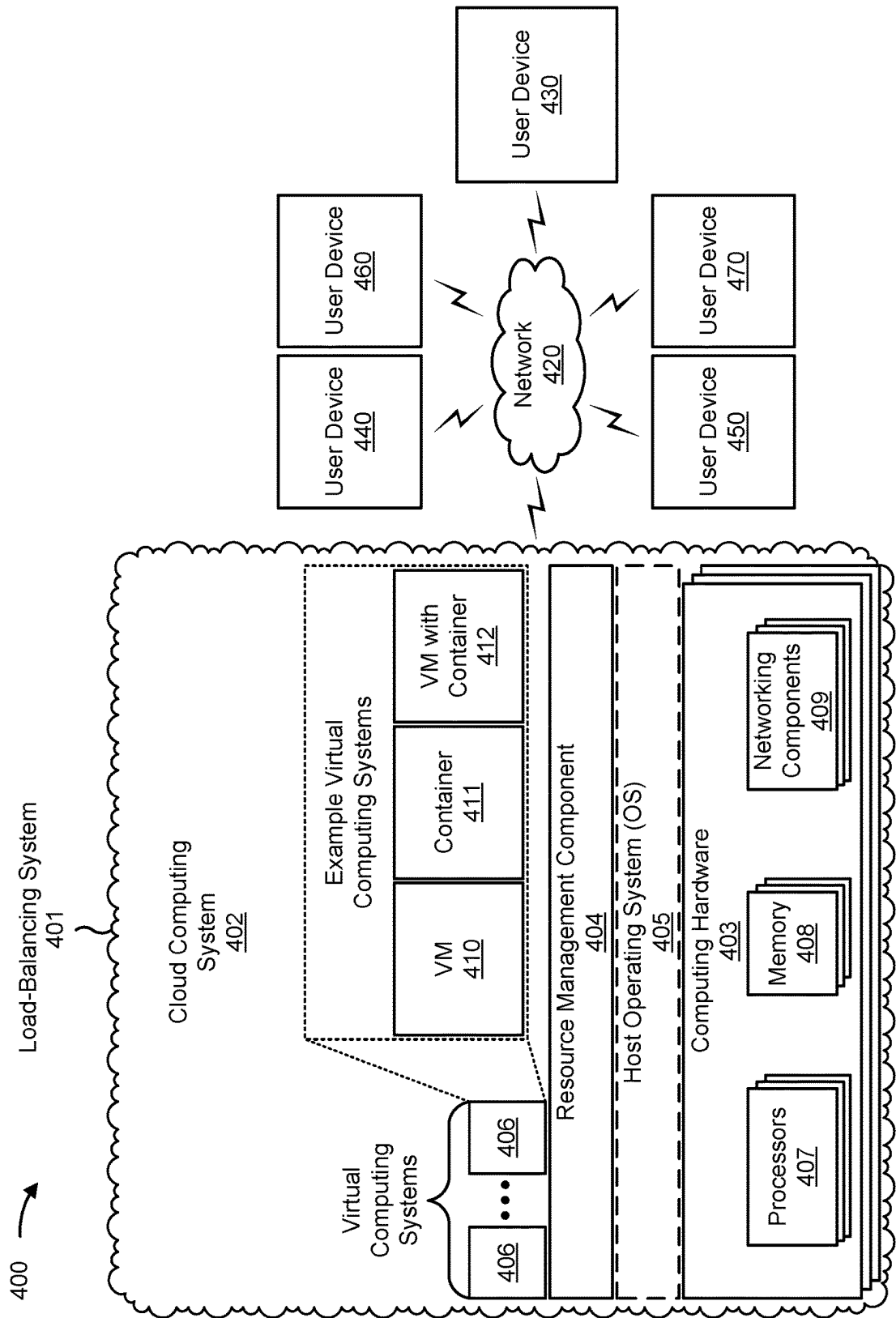


FIG. 4

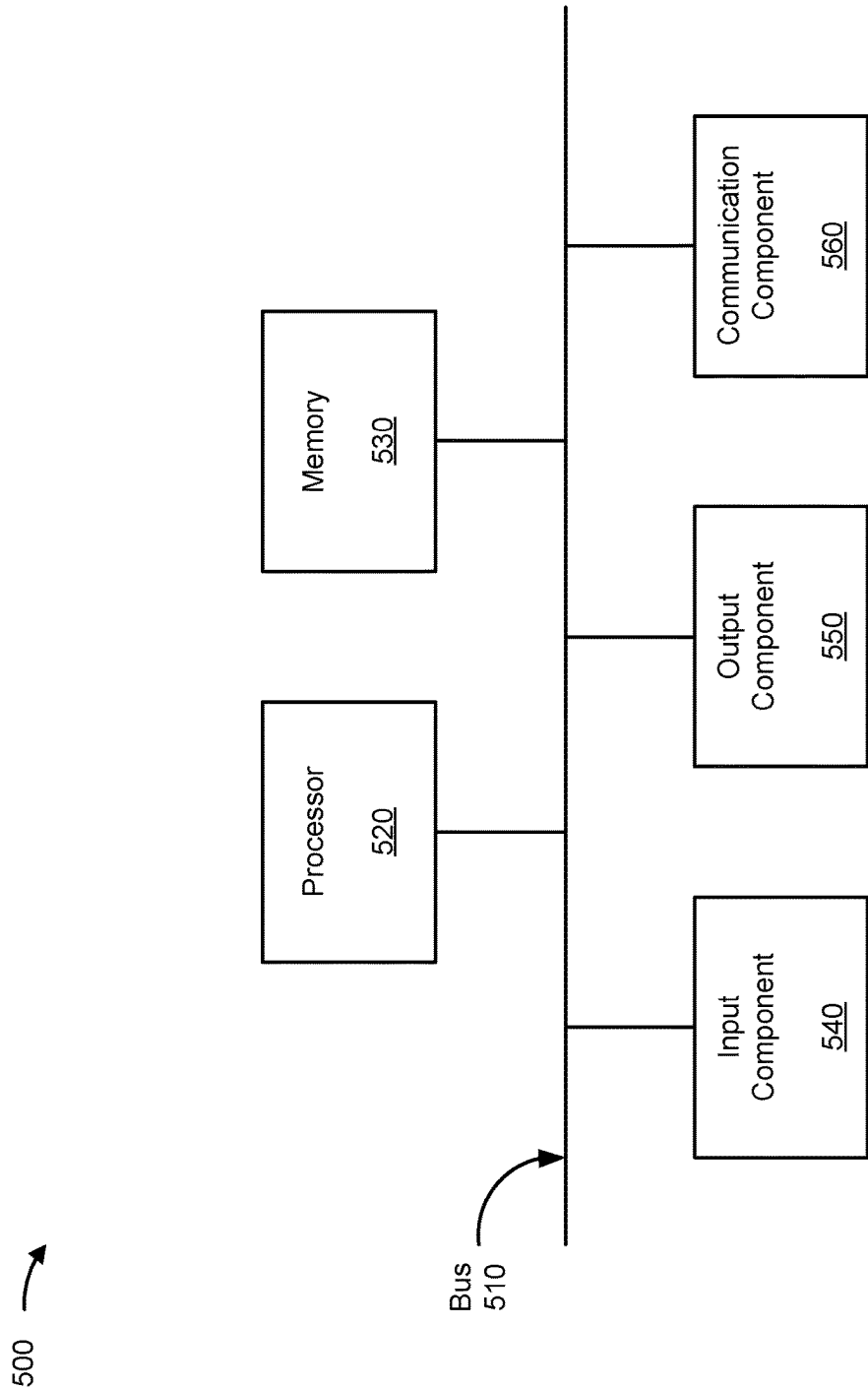


FIG. 5

600 →

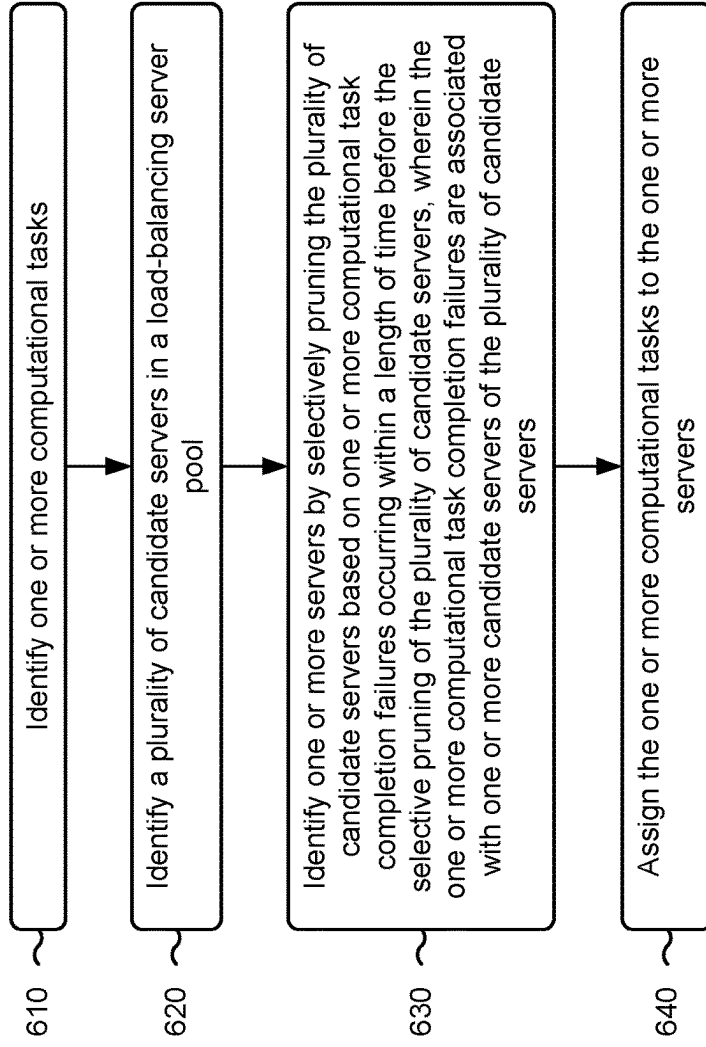


FIG. 6

## SELECTIVE PRUNING OF CANDIDATE LOAD-BALANCING SERVERS

### BACKGROUND

[0001] Load-balancing involves distributing a set of tasks over a set of resources. The tasks may be distributed among the resources based on the specific nature of the tasks, the complexity of the tasks, the hardware capabilities of the resources, error tolerance for the tasks, or the like. Load-balancing may improve the overall processing efficiency of the tasks.

### SUMMARY

[0002] Some aspects described herein relate to a system for load-balancing. The system may include one or more memories and one or more processors coupled to the one or more memories. The one or more processors may be configured to identify one or more computational tasks. The one or more processors may be configured to identify a plurality of candidate servers in a load-balancing server pool. The one or more processors may be configured to identify one or more servers by selectively pruning the plurality of candidate servers based on one or more computational task completion failures occurring within a length of time before the selective pruning of the plurality of candidate servers. The one or more computational task completion failures may be associated with one or more candidate servers of the plurality of candidate servers. The one or more processors may be configured to assign the one or more computational tasks to the one or more servers.

[0003] Some aspects described herein relate to a method of load-balancing. The method may include identifying one or more computational tasks. The method may include identifying a plurality of candidate servers in a load-balancing server pool. The method may include identifying one or more servers by selectively pruning the plurality of candidate servers based on at least one average central processing unit (CPU) utilization, associated with at least one candidate server of the plurality of candidate servers, over a quantity of CPU cycles satisfying an average CPU utilization threshold. The method may include assigning the one or more computational tasks to the one or more servers.

[0004] Some aspects described herein relate to a non-transitory computer-readable medium that stores a set of instructions by a device. The set of instructions, when executed by one or more processors of the device, may cause the device to identify one or more computational tasks. The set of instructions, when executed by one or more processors of the device, may cause the device to identify a plurality of candidate servers in a load-balancing server pool. The set of instructions, when executed by one or more processors of the device, may cause the device to identify one or more servers by selectively pruning the plurality of candidate servers based on computational task counts associated with respective candidate servers of the plurality of candidate servers. The set of instructions, when executed by one or more processors of the device, may cause the device to assign the one or more computational tasks to the one or more servers.

### BRIEF DESCRIPTION OF THE DRAWINGS

[0005] FIG. 1 is a diagram of an example associated with load-balancing one or more computational tasks, in accordance with some embodiments of the present disclosure.

[0006] FIG. 2 is a diagram of an example associated with load-balancing of computational tasks across servers, in accordance with some embodiments of the present disclosure.

[0007] FIG. 3 is a diagram of an example associated with determining a best-case server, in accordance with some embodiments of the present disclosure.

[0008] FIG. 4 is a diagram of an example environment in which systems and/or methods described herein may be implemented, in accordance with some embodiments of the present disclosure.

[0009] FIG. 5 is a diagram of example components of a device associated with selective pruning of candidate load-balancing servers, in accordance with some embodiments of the present disclosure.

[0010] FIG. 6 is a flowchart of an example process associated with selective pruning of candidate load-balancing servers, in accordance with some embodiments of the present disclosure.

### DETAILED DESCRIPTION

[0011] The following detailed description of example implementations refers to the accompanying drawings. The same reference numbers in different drawings may identify the same or similar elements.

[0012] In computing, the term “job” may refer to a unit of work or execution performed by one or more resources (e.g., servers). A job may have multiple components referred to as “tasks.” For example, one or more servers may complete a job by executing the corresponding tasks. In some examples, tasks may be placed in a queue and spawned for execution by the one or more servers. A queuing instance (e.g., a queuing server) may be responsible for managing the queuing of the tasks and related operations.

[0013] One approach to handling tasks may be referred to as “vertical scaling.” In vertical scaling, the queuing instance may also execute the tasks. For example, the queuing instance may monitor an execution state of a spawned task to identify when the task has completed (e.g., when the task has been fully executed) after which the queuing instance may begin to execute a subsequent task. Thus, vertical scaling may be a blocking process that involves singular processing of the queue (e.g., in the sense that only one task may be spawned at a time). Furthermore, by executing the tasks, the queuing instance may experience excessive resource pressure caused by running large processes that concurrently consume a limited set of resources (e.g., the queuing instance). As a result, vertical scaling may overburden the queuing instance.

[0014] In another approach to handling tasks, the queuing instance may identify a server to assign a single task based on direct assignment or by leveraging a resource utilization metric that is averaged over a small window (e.g., 1-2 minutes). As a result, the queuing instance can indiscriminately assign tasks to servers, assign entire ranges of tasks to a single system (e.g., server), or the like. Thus, this approach can lead to overloaded servers, increased run times, bypassed servers that are inactive or minimally busy, or the like.

[0015] Some implementations described herein enable dividing a list or load of tasks (e.g., subtasks) across multiple servers in parallel, which may be referred to as horizontal scaling or horizontal load-balancing. A queuing instance may identify, from among a plurality of candidate servers,

servers to assign tasks. For example, the queueing instance may identify which of the candidate servers are qualified (e.g., preferred) for handling one or more of the tasks. In some examples, the queueing instance may prune candidate servers that experienced a task completion failure within a previous time window. In some examples, the queueing instance may prune candidate servers having average CPU utilizations (e.g., averaged over a given quantity of previous CPU cycles) that exceed a target average CPU utilization. In some examples, the queueing instance may prune candidate servers that are already assigned a large quantity of computation task counts (e.g., relative to other candidate servers).

**[0016]** As a result, by leveraging multiple servers, the queueing instance may avoid becoming overburdened due to vertical scaling. Furthermore, by assigning tasks to the qualified servers, the queueing instance may help to ensure that the servers are not overloaded or underutilized and reduce run times associated with execution of the tasks. Pruning candidate servers that experienced a task completion failure within a previous time window may help to ensure that tasks are assigned to servers that are functioning correctly. Pruning candidate servers having average CPU utilizations that exceed a target average CPU utilization may help to reduce CPU utilization among the servers. Pruning candidate servers that are already assigned a large quantity of computation task counts may help to prevent servers from becoming idle or overworked.

**[0017]** FIG. 1 is a diagram of an example **100** associated with load-balancing one or more computational tasks. As shown in FIG. 1, example **100** includes a load-balancer and a load-balancing server pool. The load-balancer may be any suitable device or system configured to perform load-balancing techniques described herein. For example, the load-balancer may be, or include, a queuing server, a queueing instance, or the like. The load-balancing server pool may include a plurality of candidate servers. These devices are described in more detail in connection with FIGS. 4 and 5.

**[0018]** As shown by reference number **110**, the load-balancer may identify one or more computational tasks. The computational tasks may correspond to work to be performed by one or more servers. In some examples, the computational tasks may relate to content creation, such as text creation, image creation, or the like. For example, a large generative process (e.g., a generative artificial intelligence (AI) process) may be split into smaller units of work (e.g., the computational tasks) for handling of generative output. Upon completion of the computational tasks, the outputs of the computational tasks may be consolidated to produce a singular dataset (e.g., a single generative output). Although the computational tasks may relate to content creation in some examples, in other examples the computational tasks may relate to any suitable job.

**[0019]** In some aspects, the computational tasks may be generated automatically. For example, process-driven code (e.g., a calling process) may generate a list of the computational tasks. In some aspects, the computational tasks may be generated based on user input. For example, a user-driven process may generate a list of the computational tasks (e.g., the list of computational tasks may be generated at least in part by a user).

**[0020]** As shown by reference number **120**, the load-balancer may identify a plurality of candidate servers in the load-balancing server pool. The candidate servers may be

servers to which the load-balancer can potentially assign the computational tasks. The load-balancer may identify the plurality of candidate servers based on how the servers are designated. For example, the load-balancer may identify the plurality of candidate servers as any servers designated for data management.

**[0021]** As shown by reference number **130**, the load-balancer may identify one or more servers by selectively pruning the plurality of candidate servers. For example, the one or more servers may be a set (e.g., a subset) of the candidate servers that are qualified for handling the computational tasks. In some examples, selectively pruning the plurality of candidate servers may include pruning the plurality of candidate servers (e.g., the load-balancer may exclude at least one candidate server, which is not qualified, from the one or more servers). In some examples, selectively pruning the plurality of candidate servers may include refraining from pruning the plurality of candidate servers (e.g., in cases where all of the candidate servers are considered qualified).

**[0022]** In some aspects, the load-balancer may selectively prune the plurality of candidate servers based on one or more computational task completion failures occurring within a length of time before the selective pruning of the plurality of candidate servers. The one or more computational task completion failures may be associated with one or more candidate servers of the plurality of candidate servers. For example, the one or more computational task completion failures may be associated with one or more candidate servers in that the one or more candidate servers may have experienced the one or more computational task completion failures. For example, the load-balancer may prune any candidate servers that have experienced a failed task within the past N minutes or cycles. Thus, the load-balancer may leverage a health check to deem available a qualified server based on recent activity.

**[0023]** In some aspects, the load-balancer may selectively prune the plurality of candidate servers based on at least one average CPU utilization, associated with at least one candidate server of the plurality of candidate servers, over a quantity of CPU cycles satisfying an average CPU utilization threshold. The at least one average CPU utilization may be associated with the at least one candidate server in that the at least one candidate server may have reached the at least one average CPU utilization. For example, the load-balancer may prune any candidate servers that have a CPU utilization, averaged over the past M CPU cycles, that exceeds the average CPU utilization threshold (e.g., an appropriate maximum level). Thus, the load-balancer may identify the lowest utilizations (e.g., average CPU utilizations) to deem available a qualified server based on a recent resource (e.g., CPU) utilization of the server being within the average CPU utilization threshold.

**[0024]** In some aspects, the load-balancer may selectively prune the plurality of candidate servers based on computational task counts associated with respective candidate servers of the plurality of candidate servers. The computational task counts may be associated with respective candidate servers in that the computational task counts may be currently assigned to the respective candidate servers. For example, the load-balancer may prune any candidate servers that have a relatively high computational task count (e.g., compared to the computational task counts of other candi-

date servers). Thus, the load-balancer may leverage an availability check to prioritize servers having fewer active computational tasks.

**[0025]** In some aspects, the load-balancer may selectively prune one or more candidate servers, of the respective candidate servers, associated with nonzero computational task counts. The one or more candidate servers may be associated with nonzero computational task counts in that the one or more candidate server may have no currently assigned computational tasks. For example, the load-balancer may prune the one or more candidate servers based on the one or more candidate servers being active (e.g., having at least one currently assigned computational task). Thus, the load-balancer may prioritize inactive servers over active servers.

**[0026]** As shown by reference number **140**, the load-balancer may assign the one or more computational tasks to the one or more servers. For example, the load-balancer may assign the computational tasks to the most qualified and/or available servers. The load-balancer may assign the computational tasks after generating a list of the one or more servers.

**[0027]** In some aspects, the load-balancer may assign at least one computational task to a candidate server, of the plurality of candidate servers, that is not one of the one or more servers. For example, if no qualified (e.g., ideal) servers are available, then the load-balancer may fall back to randomly assigning computational tasks to candidate servers.

**[0028]** In some examples, the load-balancer may implement logic, referred to herein as an “infinite partitioner,” that causes the load-balancer to implement techniques described herein. For example, the load-balancer may leverage an infinite partitioner to allow for horizontal scaling while awaiting completion of each computational tasks. By leveraging an infinite partitioner, the load-balancer may help to ensure that the computational tasks are completed with a maximum degree of parallelism constrained by the quantity of like servers (e.g., candidate servers and/or best-case servers). For example, infinite partitioner may enable the load-balancer to process the generated list of computational tasks in parallel and assign a given computational task to an available server that is defined as suitable for a type of the given computational task.

**[0029]** Identifying one or more servers by selectively pruning the plurality of candidate servers may enable the load-balancer to leverage multiple servers and thereby avoid becoming overburdened due to vertical scaling. Furthermore, by assigning computational tasks to the one or more (qualified) servers, the load-balancer may help to ensure that the one or more servers are not overloaded or underutilized and may reduce run times associated with execution of the computational tasks. As a result, resource availability may be improved for additional concurrent computational tasks. Additionally, or alternatively, individual servers may require fewer resources to complete a set of computational tasks (e.g., smaller servers may be implemented). Thus, an entity (e.g., a developer, end user, or the like) may easily horizontally load-balance a list of computational tasks across multiple servers.

**[0030]** Selectively pruning the plurality of candidate servers based on the one or more computational task completion failures occurring within a length of time before the selective pruning of the plurality of candidate servers may help to

ensure that the plurality of computational tasks are assigned to servers that are functioning correctly.

**[0031]** Selectively pruning the plurality of candidate servers based on at least one average CPU utilization, associated with at least one candidate server of the plurality of candidate servers, over a quantity of CPU cycles satisfying an average CPU utilization threshold may help to reduce CPU utilization among the plurality of servers, thereby conserving CPU resources.

**[0032]** Selectively pruning the plurality of candidate servers based on computational task counts associated with respective candidate servers of the plurality of candidate servers may help to prevent servers from becoming idle or overworked.

**[0033]** Assigning at least one computational task of the plurality of computational tasks to a candidate server, of the plurality of candidate servers, that is not one of the one or more servers may help to ensure that one or more jobs corresponding to the plurality of computation tasks are completed even if no best-case servers are available to handle one or more of the computational tasks.

**[0034]** As indicated above, FIG. **1** is provided as an example. Other examples may differ from what is described with regard to FIG. **1**.

**[0035]** FIG. **2** is a diagram of an example **200** associated with load-balancing of computational tasks across servers.

**[0036]** As shown by reference number **205**, a load-balancer may identify a plurality of computational tasks. For example, the load-balancer may create a list of logical work (e.g., the computational tasks and/or corresponding jobs) to be performed. As shown by reference number **210**, the load-balancer may place the computational tasks in a process queue. As shown by reference number **215**, the load-balancer may identify a plurality of candidate servers in a load-balancing server pool (“Pool”). For example, the load-balancer may determine a quantity of candidate servers while the computational tasks are in the process queue.

**[0037]** As shown by reference number **220**, the load-balancer may place the computational tasks in a parallel process queue. As shown by reference number **225**, the load-balancer may determine the best-case server(s) (“Server 1” through “Server L”) to handle the computational tasks that are in the parallel process queue. For example, the load-balancer may selectively prune the candidate servers to identify the best-case servers as described elsewhere herein.

**[0038]** As shown by reference number **230**, the load-balancer may assign the computational tasks across the plurality of servers. For example, the load-balancer may send the computational tasks to one or more of the identified best-case servers. As shown by reference numbers **235(1)-235(L)**, servers 1-L may initiate the work (e.g., the servers 1-L may execute the computational tasks assigned by the load-balancer). When a computational task is completed, a corresponding wait block of the parallel process may be also be completed.

**[0039]** As shown by reference number **240**, upon completion of the computational tasks, the load-balancer may determine whether the parallel process queue is empty. If the parallel process queue is empty, then, as shown by reference number **245**, the process may end. If the parallel process queue is not empty, then, as shown by reference number **250**, the load-balancer may process the next work item (e.g., computational task) in the parallel process queue. For example, the load-balancer may send (e.g., assign) the next

computational task in the parallel process queue to one of the servers 1-L (e.g., a best available server).

[0040] In this manner, the load-balancer may iterate through the list of computational tasks in parallel. As each task completes, the load-balancer may remove the computational task from the list and continue through the parallel processing queue by repeating operations associated with reference numbers 225-240 until the list has been exhausted.

[0041] As indicated above, FIG. 2 is provided as an example. Other examples may differ from what is described with regard to FIG. 2.

[0042] FIG. 3 is a diagram of an example 300 associated with determining a best-case server (e.g., in accordance with reference number 225 (FIG. 2)).

[0043] As shown by reference number 305, the load-balancer may obtain a name of a queuing server. The queuing server may execute the infinite partitioner logic and assign the computational tasks from the parallel processing queue to one or more qualified servers. The queuing server may be one of the candidate servers in the load-balancing server pool or a separate server that is external to the load-balancing server pool.

[0044] As shown by reference number 310, the load-balancer may obtain a maximum average CPU utilization. For example, the load-balancer may obtain an average CPU utilization threshold. The average CPU utilization threshold may be configurable to any suitable value.

[0045] As shown by reference number 315, the load-balancer may obtain a list of like servers. For example, the load-balancer may obtain a list of the candidate servers in the load-balancing server pool. For example, the load-balancer may obtain a list of servers designated for data management that can potentially execute the computational tasks.

[0046] As shown by reference number 320, the load-balancer may obtain an indication of candidate servers that have had a computational task terminated (e.g., computational task completion failure) within the past N minutes. As shown by reference number 325, the load-balancer may remove servers with terminated computational tasks within the past N minutes from the list of like servers. For example, the load-balancer may prune the candidate servers with such terminated computational tasks.

[0047] As shown by reference number 330, the load-balancer may remove, from the list of like servers, any servers that have exceeded the maximum average CPU utilization in the past M CPU cycles. For example, the load-balancer may prune any candidate servers having an average CPU utilization over M CPU cycles that satisfies (e.g., exceeds) the average CPU utilization threshold. For example, if the average CPU utilization threshold is 70%, and a candidate server operated at 90% CPU utilization over the past M CPU cycles (e.g., 3, or any other suitable quantity, of CPU cycles), then the load-balancer may exclude that candidate server from the list of servers that are to handle the computational tasks.

[0048] As shown by reference number 335, the load-balancer may obtain an indication of active computational tasks for any remaining servers in the list of like servers. For example, the load-balancer may identify quantities of computational tasks that are currently assigned to respective candidate servers (e.g., computational task counts for the candidate servers).

[0049] As shown by reference number 340, the load-balancer may group the remaining candidate servers by server name and task count. For example, the load-balancer may sort the remaining candidate servers based on how much work the candidate servers are currently handling.

[0050] As shown by reference number 345, the load-balancer may check each group and obtain the lowest task count of the groups. As shown by reference number 350, the load-balancer may identify the server(s) with the lowest computational task count. The identified server(s) may have average CPU utilization(s) under the average CPU utilization threshold.

[0051] As shown by reference number 355, the load-balancer may count the groups. As shown by reference number 360, if the count of groups is less than the count of remaining (e.g., non-pruned) servers, then the load-balancer may exclude active servers to prioritize the inactive servers. The active servers may be those with computational tasks currently assigned, and inactive servers may have no computational tasks currently assigned. For example, the load-balancer may prune one or more candidate servers associated with nonzero computational task counts.

[0052] As shown by reference number 365, the load-balancer may return the last server in the list or return an empty string to force default queuing. If the list contains at least one server after the pruning of candidate servers, then the load-balancer may return one or more servers (including the last server in the list). The last server in the list may be the best-case server (e.g., the server with the lowest computational task count that also has an average CPU utilization under the average CPU utilization threshold and has had no computational tasks terminated within the past N minutes). If the load-balancer, returns an empty string, then there are no remaining servers in the list, and the load-balancer may fall back to default queuing (e.g., by assigning computational tasks to candidate servers).

[0053] As indicated above, FIG. 3 is provided as an example. Other examples may differ from what is described with regard to FIG. 3.

[0054] FIG. 4 is a diagram of an example environment 400 in which systems and/or methods described herein may be implemented. As shown in FIG. 4, environment 400 may include a load-balancing system 401, which may include one or more elements of and/or may execute within a cloud computing system 402. The cloud computing system 402 may include one or more elements 403-412, as described in more detail below. As further shown in FIG. 4, environment 400 may include a network 420, and/or one or more of user devices 430-470. Devices and/or elements of environment 400 may interconnect via wired connections and/or wireless connections.

[0055] The cloud computing system 402 may include computing hardware 403, a resource management component 404, a host operating system (OS) 405, and/or one or more virtual computing systems 406. The cloud computing system 402 may execute on, for example, an Amazon Web Services platform, a Microsoft Azure platform, or a Snowflake platform. The resource management component 404 may perform virtualization (e.g., abstraction) of computing hardware 403 to create the one or more virtual computing systems 406. Using virtualization, the resource management component 404 enables a single computing device (e.g., a computer or a server) to operate like multiple computing devices, such as by creating multiple isolated virtual com-

puting systems **406** from computing hardware **403** of the single computing device. In this way, computing hardware **403** can operate more efficiently, with lower power consumption, higher reliability, higher availability, higher utilization, greater flexibility, and lower cost than using separate computing devices.

[0056] The computing hardware **403** may include hardware and corresponding resources from one or more computing devices. For example, computing hardware **403** may include hardware from a single computing device (e.g., a single server) or from multiple computing devices (e.g., multiple servers), such as multiple computing devices in one or more data centers. As shown, computing hardware **403** may include one or more processors **407**, one or more memories **408**, and/or one or more networking components **409**. Examples of a processor, a memory, and a networking component (e.g., a communication component) are described elsewhere herein.

[0057] The resource management component **404** may include a virtualization application (e.g., executing on hardware, such as computing hardware **403**) capable of virtualizing computing hardware **403** to start, stop, and/or manage one or more virtual computing systems **406**. For example, the resource management component **404** may include a hypervisor (e.g., a bare-metal or Type 1 hypervisor, a hosted or Type 2 hypervisor, or another type of hypervisor) or a virtual machine monitor, such as when the virtual computing systems **406** are virtual machines **410**. Additionally, or alternatively, the resource management component **404** may include a container manager, such as when the virtual computing systems **406** are containers **411**. In some implementations, the resource management component **404** executes within and/or in coordination with a host operating system **405**.

[0058] A virtual computing system **406** may include a virtual environment that enables cloud-based execution of operations and/or processes described herein using computing hardware **403**. As shown, a virtual computing system **406** may include a virtual machine **410**, a container **411**, or a hybrid environment **412** that includes a virtual machine and a container, among other examples. A virtual computing system **406** may execute one or more applications using a file system that includes binary files, software libraries, and/or other resources required to execute applications on a guest operating system (e.g., within the virtual computing system **406**) or the host operating system **405**.

[0059] Although the load-balancing system **401** may include one or more elements **403-412** of the cloud computing system **402**, may execute within the cloud computing system **402**, and/or may be hosted within the cloud computing system **402**, in some implementations, the load-balancing system **401** may not be cloud-based (e.g., may be implemented outside of a cloud computing system) or may be partially cloud-based. For example, the load-balancing system **401** may include one or more devices that are not part of the cloud computing system **402**, such as device **500** of FIG. 5, which may include a standalone server or another type of computing device. The load-balancing system **401** may perform one or more operations and/or processes described in more detail elsewhere herein.

[0060] The network **420** may include one or more wired and/or wireless networks. For example, the network **420** may include a cellular network, a public land mobile network (PLMN), a local area network (LAN), a wide area

network (WAN), a private network, the Internet, and/or a combination of these or other types of networks. The network **420** enables communication among the devices of the environment **400**.

[0061] The user devices **430-470** may include one or more devices capable of receiving, generating, storing, processing, and/or providing information associated with load-balancing, as described elsewhere herein. The user devices **430-470** may each include a communication device and/or a computing device. For example, the user devices **430-470** may include a wireless communication device, a mobile phone, a user equipment, a laptop computer, a tablet computer, a desktop computer, a gaming console, a set-top box, a wearable communication device (e.g., a smart wristwatch, a pair of smart eyeglasses, a head mounted display, or a virtual reality headset), or a similar type of device.

[0062] The number and arrangement of devices and networks shown in FIG. 4 are provided as an example. In practice, there may be additional devices and/or networks, fewer devices and/or networks, different devices and/or networks, or differently arranged devices and/or networks than those shown in FIG. 4. Furthermore, two or more devices shown in FIG. 4 may be implemented within a single device, or a single device shown in FIG. 4 may be implemented as multiple, distributed devices. Additionally, or alternatively, a set of devices (e.g., one or more devices) of the environment **400** may perform one or more functions described as being performed by another set of devices of the environment **400**.

[0063] FIG. 5 is a diagram of example components of a device **500** associated with selective pruning of candidate load-balancing servers. The device **500** may correspond to the load-balancing system **401**. In some implementations, the load-balancing system **401** may include one or more devices **500** and/or one or more components of the device **500**. As shown in FIG. 5, the device **500** may include a bus **510**, a processor **520**, a memory **530**, an input component **540**, an output component **550**, and/or a communication component **560**.

[0064] The bus **510** may include one or more components that enable wired and/or wireless communication among the components of the device **500**. The bus **510** may couple together two or more components of FIG. 5, such as via operative coupling, communicative coupling, electronic coupling, and/or electric coupling. For example, the bus **510** may include an electrical connection (e.g., a wire, a trace, and/or a lead) and/or a wireless bus. The processor **520** may include a central processing unit, a graphics processing unit, a microprocessor, a controller, a microcontroller, a digital signal processor, a field-programmable gate array, an application-specific integrated circuit, and/or another type of processing component. The processor **520** may be implemented in hardware, firmware, or a combination of hardware and software. In some implementations, the processor **520** may include one or more processors capable of being programmed to perform one or more operations or processes described elsewhere herein.

[0065] The memory **530** may include volatile and/or non-volatile memory. For example, the memory **530** may include random access memory (RAM), read only memory (ROM), a hard disk drive, and/or another type of memory (e.g., a flash memory, a magnetic memory, and/or an optical memory). The memory **530** may include internal memory (e.g., RAM, ROM, or a hard disk drive) and/or removable

memory (e.g., removable via a universal serial bus connection). The memory 530 may be a non-transitory computer-readable medium. The memory 530 may store information, one or more instructions, and/or software (e.g., one or more software applications) related to the operation of the device 500. In some implementations, the memory 530 may include one or more memories that are coupled (e.g., communicatively coupled) to one or more processors (e.g., processor 520), such as via the bus 510. Communicative coupling between a processor 520 and a memory 530 may enable the processor 520 to read and/or process information stored in the memory 530 and/or to store information in the memory 530.

[0066] The input component 540 may enable the device 500 to receive input, such as user input and/or sensed input. For example, the input component 540 may include a touch screen, a keyboard, a keypad, a mouse, a button, a microphone, a switch, a sensor, a global positioning system sensor, a global navigation satellite system sensor, an accelerometer, a gyroscope, and/or an actuator. The output component 550 may enable the device 500 to provide output, such as via a display, a speaker, and/or a light-emitting diode. The communication component 560 may enable the device 500 to communicate with other devices via a wired connection and/or a wireless connection. For example, the communication component 560 may include a receiver, a transmitter, a transceiver, a modem, a network interface card, and/or an antenna.

[0067] The device 500 may perform one or more operations or processes described herein. For example, a non-transitory computer-readable medium (e.g., memory 530) may store a set of instructions (e.g., one or more instructions or code) for execution by the processor 520. The processor 520 may execute the set of instructions to perform one or more operations or processes described herein. In some implementations, execution of the set of instructions, by one or more processors 520, causes the one or more processors 520 and/or the device 500 to perform one or more operations or processes described herein. In some implementations, hardwired circuitry may be used instead of or in combination with the instructions to perform one or more operations or processes described herein. Additionally, or alternatively, the processor 520 may be configured to perform one or more operations or processes described herein. Thus, implementations described herein are not limited to any specific combination of hardware circuitry and software.

[0068] The number and arrangement of components shown in FIG. 5 are provided as an example. The device 500 may include additional components, fewer components, different components, or differently arranged components than those shown in FIG. 5. Additionally, or alternatively, a set of components (e.g., one or more components) of the device 500 may perform one or more functions described as being performed by another set of components of the device 500.

[0069] FIG. 6 is a flowchart of an example process 600 associated with selective pruning of candidate load-balancing servers. In some implementations, one or more process blocks of FIG. 6 may be performed by the load-balancing system 401. In some implementations, one or more process blocks of FIG. 6 may be performed by another device or a group of devices separate from or including the load-balancing system 401, such as the user devices 430-470. Additionally, or alternatively, one or more process blocks of

FIG. 6 may be performed by one or more components of the device 500, such as processor 520, memory 530, input component 540, output component 550, and/or communication component 560.

[0070] As shown in FIG. 6, process 600 may include identifying one or more computational tasks (block 610). For example, the load-balancing system 401 (e.g., using processor 520 and/or memory 530) may identify one or more computational tasks, as described above in connection with reference number 110 of FIG. 1. As an example, the computational tasks may correspond to work to be performed by one or more servers.

[0071] As further shown in FIG. 6, process 600 may include identifying a plurality of candidate servers in a load-balancing server pool (block 620). For example, the load-balancing system 401 (e.g., using processor 520 and/or memory 530) may identify a plurality of candidate servers in a load-balancing server pool, as described above in connection with reference number 120 of FIG. 1. As an example, the candidate servers may be servers to which the load-balancer can potentially assign the computational tasks.

[0072] As further shown in FIG. 6, process 600 may include identifying one or more servers by selectively pruning the plurality of candidate servers based on one or more computational task completion failures occurring within a length of time before the selective pruning of the plurality of candidate servers, wherein the one or more computational task completion failures are associated with one or more candidate servers of the plurality of candidate servers (block 630). For example, the load-balancing system 401 (e.g., using processor 520 and/or memory 530) may identify one or more servers by selectively pruning the plurality of candidate servers based on one or more computational task completion failures occurring within a length of time before the selective pruning of the plurality of candidate servers, wherein the one or more computational task completion failures are associated with one or more candidate servers of the plurality of candidate servers, as described above in connection with reference number 130 of FIG. 1. As an example, the load-balancer may prune any candidate servers that have experienced a failed task within the past N minutes.

[0073] As further shown in FIG. 6, process 600 may include assigning the one or more computational tasks to the one or more servers (block 640). For example, the load-balancing system 401 (e.g., using processor 520 and/or memory 530) may assign the one or more computational tasks to the one or more servers, as described above in connection with reference number 140 of FIG. 1. As an example, the load-balancer may assign the computational tasks to the most qualified and/or available servers that have not been pruned.

[0074] Although FIG. 6 shows example blocks of process 600, in some implementations, process 600 may include additional blocks, fewer blocks, different blocks, or differently arranged blocks than those depicted in FIG. 6. Additionally, or alternatively, two or more of the blocks of process 600 may be performed in parallel. The process 600 is an example of one process that may be performed by one or more devices described herein. These one or more devices may perform one or more other processes based on operations described herein, such as the operations described in connection with FIGS. 1-3. Moreover, while the process 600 has been described in relation to the devices and components

of the preceding figures, the process 600 can be performed using alternative, additional, or fewer devices and/or components. Thus, the process 600 is not limited to being performed with the example devices, components, hardware, and software explicitly enumerated in the preceding figures.

**[0075]** The foregoing disclosure provides illustration and description, but is not intended to be exhaustive or to limit the implementations to the precise forms disclosed. Modifications may be made in light of the above disclosure or may be acquired from practice of the implementations.

**[0076]** As used herein, the term “component” is intended to be broadly construed as hardware, firmware, or a combination of hardware and software. It will be apparent that systems and/or methods described herein may be implemented in different forms of hardware, firmware, and/or a combination of hardware and software. The hardware and/or software code described herein for implementing aspects of the disclosure should not be construed as limiting the scope of the disclosure. Thus, the operation and behavior of the systems and/or methods are described herein without reference to specific software code—it being understood that software and hardware can be used to implement the systems and/or methods based on the description herein.

**[0077]** As used herein, satisfying a threshold may, depending on the context, refer to a value being greater than the threshold, greater than or equal to the threshold, less than the threshold, less than or equal to the threshold, equal to the threshold, not equal to the threshold, or the like.

**[0078]** Although particular combinations of features are recited in the claims and/or disclosed in the specification, these combinations are not intended to limit the disclosure of various implementations. In fact, many of these features may be combined in ways not specifically recited in the claims and/or disclosed in the specification. Although each dependent claim listed below may directly depend on only one claim, the disclosure of various implementations includes each dependent claim in combination with every other claim in the claim set. As used herein, a phrase referring to “at least one of” a list of items refers to any combination and permutation of those items, including single members. As an example, “at least one of: a, b, or c” is intended to cover a, b, c, a-b, a-c, b-c, and a-b-c, as well as any combination with multiple of the same item. As used herein, the term “and/or” used to connect items in a list refers to any combination and any permutation of those items, including single members (e.g., an individual item in the list). As an example, “a, b, and/or c” is intended to cover a, b, c, a-b, a-c, b-c, and a-b-c.

**[0079]** When “a processor” or “one or more processors” (or another device or component, such as “a controller” or “one or more controllers”) is described or claimed (within a single claim or across multiple claims) as performing multiple operations or being configured to perform multiple operations, this language is intended to broadly cover a variety of processor architectures and environments. For example, unless explicitly claimed otherwise (e.g., via the use of “first processor” and “second processor” or other language that differentiates processors in the claims), this language is intended to cover a single processor performing or being configured to perform all of the operations, a group of processors collectively performing or being configured to perform all of the operations, a first processor performing or being configured to perform a first operation and a second

processor performing or being configured to perform a second operation, or any combination of processors performing or being configured to perform the operations. For example, when a claim has the form “one or more processors configured to: perform X; perform Y; and perform Z,” that claim should be interpreted to mean “one or more processors configured to perform X; one or more (possibly different) processors configured to perform Y; and one or more (also possibly different) processors configured to perform Z.”

**[0080]** No element, act, or instruction used herein should be construed as critical or essential unless explicitly described as such. Also, as used herein, the articles “a” and “an” are intended to include one or more items, and may be used interchangeably with “one or more.” Further, as used herein, the article “the” is intended to include one or more items referenced in connection with the article “the” and may be used interchangeably with “the one or more.” Furthermore, as used herein, the term “set” is intended to include one or more items (e.g., related items, unrelated items, or a combination of related and unrelated items), and may be used interchangeably with “one or more.” Where only one item is intended, the phrase “only one” or similar language is used. Also, as used herein, the terms “has,” “have,” “having,” or the like are intended to be open-ended terms. Further, the phrase “based on” is intended to mean “based, at least in part, on” unless explicitly stated otherwise. Also, as used herein, the term “or” is intended to be inclusive when used in a series and may be used interchangeably with “and/or,” unless explicitly stated otherwise (e.g., if used in combination with “either” or “only one of”).

1. A system for load-balancing, the system comprising:

one or more memories; and

one or more processors, communicatively coupled to the one or more memories, configured to:

identify one or more computational tasks;

identify a plurality of candidate servers in a load-balancing server pool;

identify one or more servers by selectively pruning the plurality of candidate servers based on one or more computational task completion failures occurring within a length of time before the selective pruning of the plurality of candidate servers, wherein the one or more computational task completion failures are associated with one or more candidate servers of the plurality of candidate servers;

identify the one or more servers by selectively pruning the plurality of candidate servers based further on computational task counts associated with respective candidate servers of the plurality of candidate servers; and

assign the one or more computational tasks to the one or more servers.

2. The system of claim 1, wherein the one or more processors, to identify the one or more servers, are configured to:

identify the one or more servers by selectively pruning the plurality of candidate servers based further on at least one average central processing unit (CPU) utilization, associated with at least one candidate server of the plurality of candidate servers, over a quantity of CPU cycles satisfying an average CPU utilization threshold.

3. (canceled)
4. The system of claim 1, wherein the one or more processors, to identify the one or more servers, are configured to:
  - identify the one or more servers by selectively pruning one or more candidate servers, of the respective candidate servers, associated with nonzero computational task counts.
5. The system of claim 1, wherein the one or more processors are further configured to:
  - assign at least one computational task to a candidate server, of the plurality of candidate servers, that is not one of the one or more servers.
6. The system of claim 1, wherein the one or more computational tasks are generated automatically.
7. The system of claim 1, wherein the one or more computational tasks are generated based on user input.
8. A method of load-balancing, comprising:
  - identifying one or more computational tasks;
  - identifying a plurality of candidate servers in a load-balancing server pool;
  - identifying one or more servers by selectively pruning the plurality of candidate servers based on at least one average central processing unit (CPU) utilization, associated with at least one candidate server of the plurality of candidate servers, over a quantity of CPU cycles satisfying an average CPU utilization threshold; and
  - assigning the one or more computational tasks to the one or more servers.
9. The method of claim 8, wherein identifying the one or more servers further includes:
  - identifying the one or more servers by selectively pruning the plurality of candidate servers based further on one or more computational task completion failures occurring within a length of time before the selective pruning of the plurality of candidate servers, wherein the one or more computational task completion failures are associated with one or more candidate servers of the plurality of candidate servers.
10. The method of claim 8, wherein identifying the one or more servers further includes:
  - identifying the one or more servers by selectively pruning the plurality of candidate servers based further on computational task counts associated with respective candidate servers of the plurality of candidate servers.
11. The method of claim 10, wherein identifying the one or more servers further includes:
  - identifying the one or more servers by selectively pruning one or more candidate servers, of the respective candidate servers, associated with nonzero computational task counts.
12. The method of claim 8, further comprising:
  - assigning at least one computational task to a candidate server, of the plurality of candidate servers, that is not one of the one or more servers.
13. The method of claim 8, wherein the one or more computational tasks are generated automatically.
14. The method of claim 8, wherein the one or more computational tasks are generated based on user input.
15. A non-transitory computer-readable medium storing a set of instructions, the set of instructions comprising:
  - one or more instructions that, when executed by one or more processors of a device, cause the device to:
    - identify one or more computational tasks;
    - identify a plurality of candidate servers in a load-balancing server pool;
    - identify one or more servers by selectively pruning the plurality of candidate servers based on computational task counts associated with respective candidate servers of the plurality of candidate servers; and
    - assign the one or more computational tasks to the one or more servers.
16. The non-transitory computer-readable medium of claim 15, wherein the one or more instructions, that cause the device to identify the one or more servers, cause the device to:
  - identify the one or more servers by selectively pruning the plurality of candidate servers based further on one or more computational task completion failures occurring within a length of time before the selective pruning of the plurality of candidate servers, wherein the one or more computational task completion failures are associated with one or more candidate servers of the plurality of candidate servers.
17. The non-transitory computer-readable medium of claim 15, wherein the one or more instructions, that cause the device to identify the one or more servers, cause the device to:
  - identify the one or more servers by selectively pruning the plurality of candidate servers based further on at least one average central processing unit (CPU) utilization, associated with at least one candidate server of the plurality of candidate servers, over a quantity of CPU cycles satisfying an average CPU utilization threshold.
18. The non-transitory computer-readable medium of claim 15, wherein the one or more instructions, that cause the device to identify the one or more servers, cause the device to:
  - identify the one or more servers by selectively pruning one or more candidate servers, of the respective candidate servers, associated with nonzero computational task counts.
19. The non-transitory computer-readable medium of claim 15, wherein the one or more instructions, when executed by the one or more processors, further cause the device to:
  - assign at least one computational task to a candidate server, of the plurality of candidate servers, that is not one of the one or more servers.
20. The non-transitory computer-readable medium of claim 15, wherein the one or more computational tasks are generated based on user input.
21. The system of claim 1, wherein the one or more processors, to identify the one or more servers, are configured to:
  - identifying the one or more servers by selectively pruning one or more candidate servers, of the respective candidate servers, based on the one or more candidate servers having at least one currently assigned computational task.

\* \* \* \* \*